

Lista 1

Introdução à Ciência de Computação II (SCC0201)

Professor: Dr. Marcelo Garcia Manzato

Monitores: Gustavo Lima Lopes e Lucas Lombardi Castro

Assuntos: Revisão C e Recursão

Exercícios Teóricos

1. Na linguagem C, qual a função dos operadores `*` e `&` ao trabalhar com ponteiros?

Leia o código abaixo e responda às próximas três perguntas. Considere que a função `malloc` retornou com sucesso o endereço de memória `0x16d3b6a6c`.

```
1. int i = 1;
2. int *ptr = (int*) malloc(sizeof(int));
3. if(ptr == NULL) return 1;
4. *ptr = i;
5. if(&ptr == &i) {
6.     printf("Eles possuem o mesmo endereco!\n");
7. }
8. free(ptr);
```

2. Assinale a alternativa correta sobre o que está ocorrendo no código:
 - a) O ponteiro está armazenando o valor `'1'` diretamente, e não um endereço de memória.
 - b) O código aloca uma região de memória dinamicamente, mas o acesso a essa região na linha 4 está incorreto pela falta do operador `&`.
 - c) Uma região de memória é alocada dinamicamente para armazenar um inteiro (4 bytes). O ponteiro `ptr` armazena o endereço dessa região e, na linha 4, o valor contido nesse endereço é alterado para o valor de `i`.
3. Qual é o respectivo valor armazenado no ponteiro `ptr` e o conteúdo apontado por ele logo após a execução da linha 4?
 - a) `0x16d3b6a6c` e `1`.
 - b) `1` e `0x16d3b6a6c`.
 - c) `0` e `0x16d3b6a6c`.
 - d) `0x16d3b6a6c` e `0`.
4. O comando `printf` da linha 6 será executado? Ou seja, o `if` da linha 5 retornará `True` ou `False`? Por quê?

5. Analise a função recursiva abaixo:

```
int computa_indice(int n) {  
    if (n == 0) {  
        return 0;  
    }  
    return n + computa_indice(n - 2);  
}
```

O que acontecerá se fizermos a chamada `computa_indice(5)` na função `main`?

- a) A função retornará 9, pois somará os ímpares ($5 + 3 + 1$).
- b) A função retornará 8, pois somará apenas ($5 + 3$), parando antes do 1.
- c) O programa causará um erro de estouro de pilha (Stack Overflow), pois a condição de parada `n == 0` nunca será satisfeita, resultando em chamadas infinitas.
- d) O compilador emitirá um erro, pois uma função recursiva deve sempre subtrair exatamente 1 em sua chamada.

6. Observe o código a seguir:

```
#include <stdio.h>  
  
void exibe_niveis(int n) {  
    if (n == 0) {  
        return;  
    }  
    exibe_niveis(n - 1);  
    printf("%d ", n);  
}  
  
int main() {  
    exibe_niveis(3);  
    return 0;  
}
```

Assinale a alternativa que indica a exata saída impressa no terminal ao executar o código acima:

- a) 3 2 1
- b) 1 2 3
- c) 0 1 2 3
- d) 3 2 1 0

Exercício Prático (Desafio)

7. Inversão de String In-Place (Ponteiros + Recursão)

Escreva uma função recursiva em C que inverta os caracteres de uma string diretamente na memória original (*in-place*).

Regras:

- A função deve receber exatamente dois ponteiros para `char`: um apontando para o caractere atual à esquerda e outro para o caractere atual à direita.
- Você **não** pode usar índices de vetor (ex: `str[i]`). Utilize apenas aritmética de ponteiros e desreferenciação (*).
- Nenhuma função da `<string.h>` pode ser chamada dentro da recursão.
- Nenhuma estrutura auxiliar (como novos arrays) deve ser alocada.

A assinatura da função deve ser:

```
void inverte_texto_rec(char *esq, char *dir);
```

Dica: Antes de iniciar, pense com cuidado, qual é o **caso base** dessa recursão considerando os endereços de memória dos ponteiros? O que deve ser feito no passo recursivo?